The advantage of this sort of integration is that releases need only be managed in one place. It also makes it easier for release managers to prepare release notes based on the current state of the automated acceptance criteria.

You don't have to stick with a single approach for your living documentation. In practice, teams often use a mix of several strategies for different purposes or different audiences. For example, a requirements-based organization is more effective for documenting what the application does as a whole and how it does it, whereas release-focused reporting is more relevant when reporting on progress and preparing release notes.

11.5 Providing more free-form documentation

Living documentation should aim to be just that: documentation. It should be organized in a way that makes it easy to use and simple to find the information you're looking for. It should also contain enough information to make it useful, albeit lightweight, documentation.

A user story is typically described as a "placeholder for a conversation," with a deliberate minimum of information. But when the acceptance criteria are fleshed out, teams often find it useful to have a little more descriptive text around a feature and story than just the "in order to ... as a ... I want" text. This can include more background about the requirement, references to more detailed business rules, legislative requirements, and so forth.

There are several ways to provide this sort of additional information. In Cucumber and the other Gherkin-based tools, for example, the description section of a feature file is effectively free text, so teams often put quite detailed descriptive texts directly in the feature file.

Teams integrating with an electronic backlog system often store more detailed documentation in the backlog itself. This may even include links to external sources such as an enterprise wiki for additional details or documents. When using a reporting tool like Thucydides, the description text in the electronic backlog will appear in the living documentation, including any hyperlinks, so no information is lost in the reporting process.

Some automated acceptance-testing tools such as FitNesse and Concordion (http://concordion.org) give you even more flexibility in how you organize the living documentation. FitNesse (http://fitnesse.org/) uses wiki pages to let the team, including business analysts and even users, write their own acceptance criteria in a tabular format (see figure 11.11).

Developers then implement the corresponding acceptance tests using "fixture" classes, typically in Java. These classes interact with the wiki tables and call the actual production code to perform the tests. For example, a fixture class for the scenario in figure 11.11 might look like this:

```
public class StatusBoundaries extends ColumnFixture {
    public String initialStatus;
    public int initialStatusPoints;
    public int extraPoints;
```